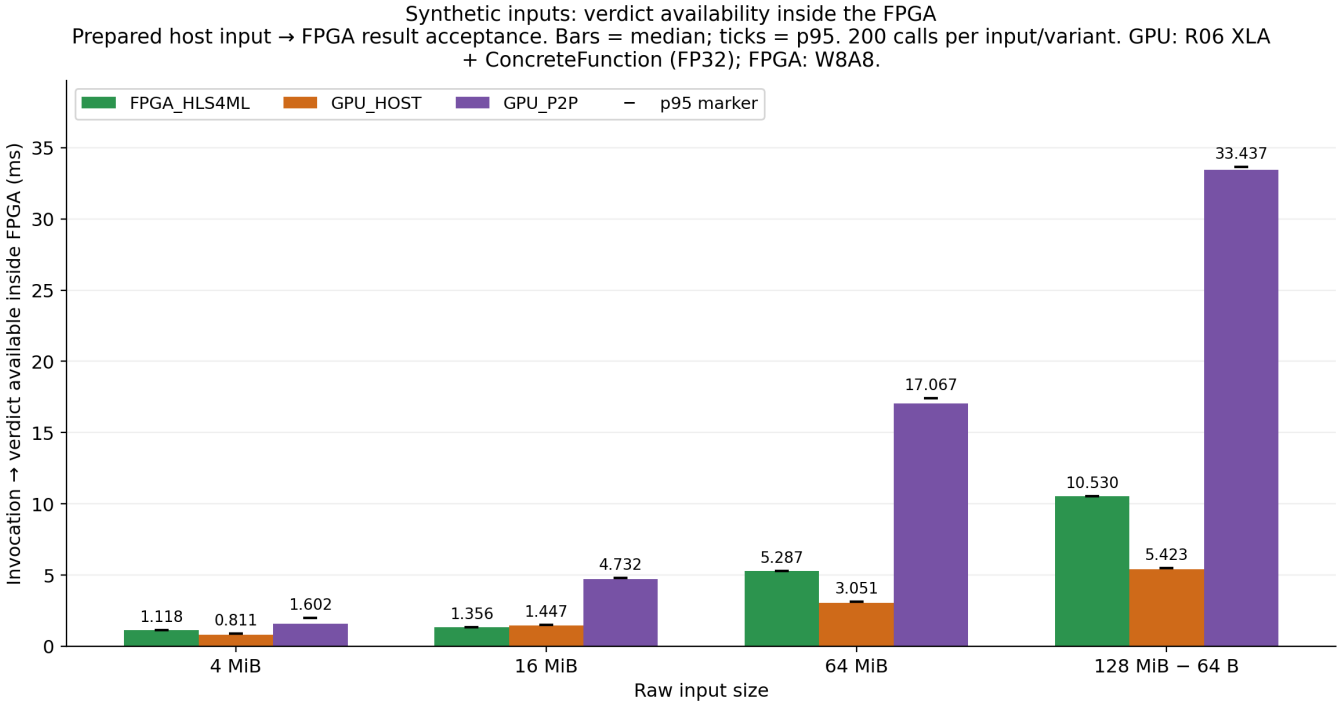
hls4ml-checker versus R06 GPU inference

Figure 1 · Synthetic inputs · GPU: XLA + ConcreteFunction (R06)

Report: **22 October 2026** · Accepted measurements: 21 October 2026.

The bitstream is already prepared in host RAM when timing starts. Timing ends when the FPGA accepts the classification result: the local CNN score for FPGA_HLS4ML, or the validated GPU score for GPU_HOST and GPU_P2P. This is the existing **CSR14 timestamp**.



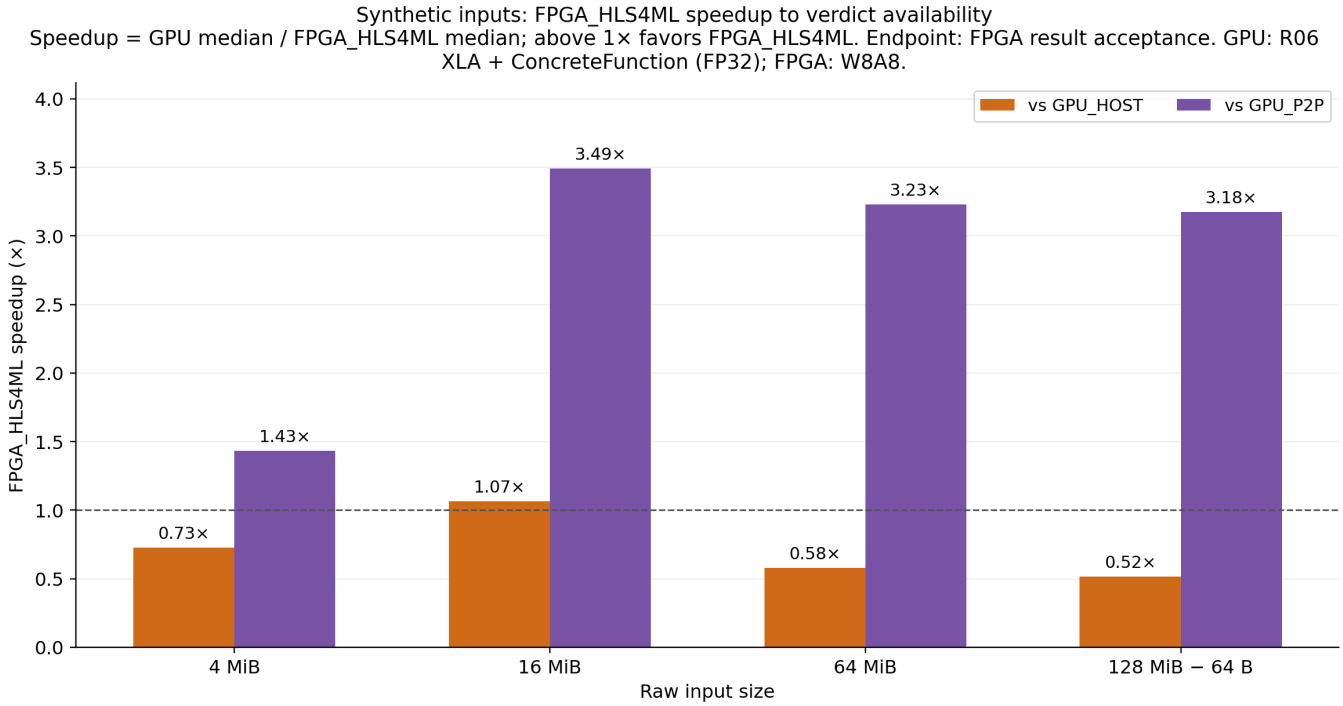
At 4 MiB: FPGA_HLS4ML takes 1.118 ms, GPU_HOST 0.811 ms, and GPU_P2P 1.602 ms. At 128 MiB minus 64 B, the medians are 10.530, 5.423, and 33.437 ms.

GPU_HOST is faster at 4, 64 and approximately 128 MiB. FPGA_HLS4ML is slightly faster at 16 MiB, and faster than GPU_P2P at all four sizes. Bars show medians; black ticks show p95 across 200 calls per input and implementation. GPU_P2P includes autonomous polling and result retrieval. Every call still transfers the complete input, even when the result reaches the FPGA earlier.

Endpoint distinction: accepting the classification result does not imply that the final verdict register has been updated or that reconfiguration may begin. Figures 4 and 5 show input arrival separately. FPGA uses W8A8; GPU uses the original FP32 model with R06. Compilation and warmup are excluded. Existing FPGA results are retained; the shared streaming control changes by at most 0.028% across campaigns. Real bitstreams appear only in the appendix.

Checker speedup to verdict availability

Figure 2 · Synthetic inputs · Values above 1 favor FPGA_HLS4ML



Speedup = GPU median result-availability latency / FPGA_HLS4ML median result-availability latency. Both timings end inside the FPGA at acceptance of the classification result. Values below 1 favor the GPU path.

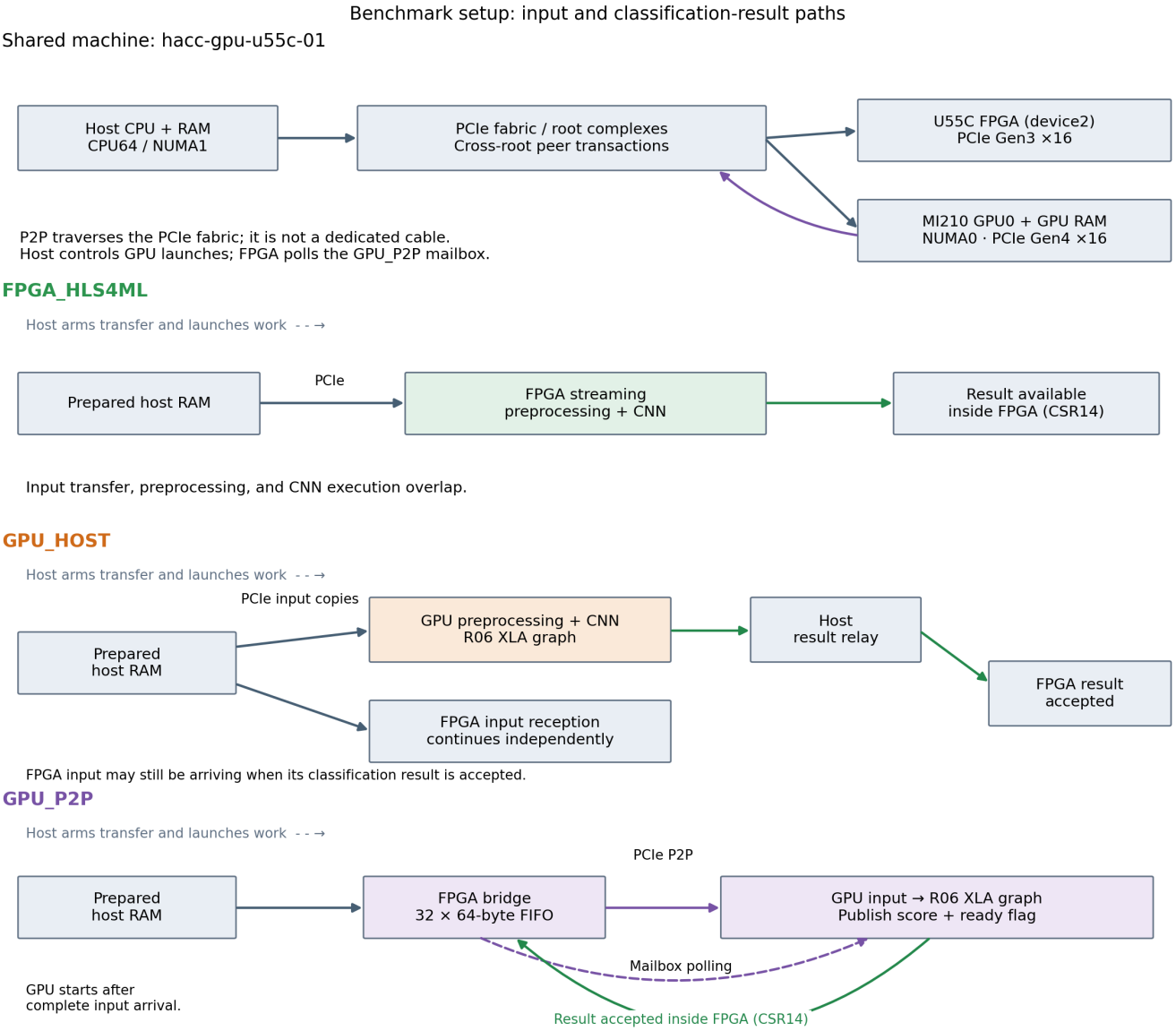
Against GPU_HOST, the ratios are 0.73x at 4 MiB and 0.52x at the largest input. Against GPU_P2P, they are 1.43x and 3.18x.

R06 combines preprocessing and inference in a static-length XLA graph and calls its cached ConcreteFunction directly. GPU_HOST can deliver its result before the FPGA receives all input bytes. Its Gen4 x16 GPU copy is faster than reception through the FPGA Gen3 x16 path. The revised endpoint exposes this difference instead of including the subsequent wait for the remaining FPGA input.

Ratios use independent-run medians, not paired trials or confidence intervals. These compare complete implemented paths with different model precisions; they are not equal-precision kernel benchmarks.

Benchmark setup and data paths

Figure 3 · Shared physical setup and logical paths

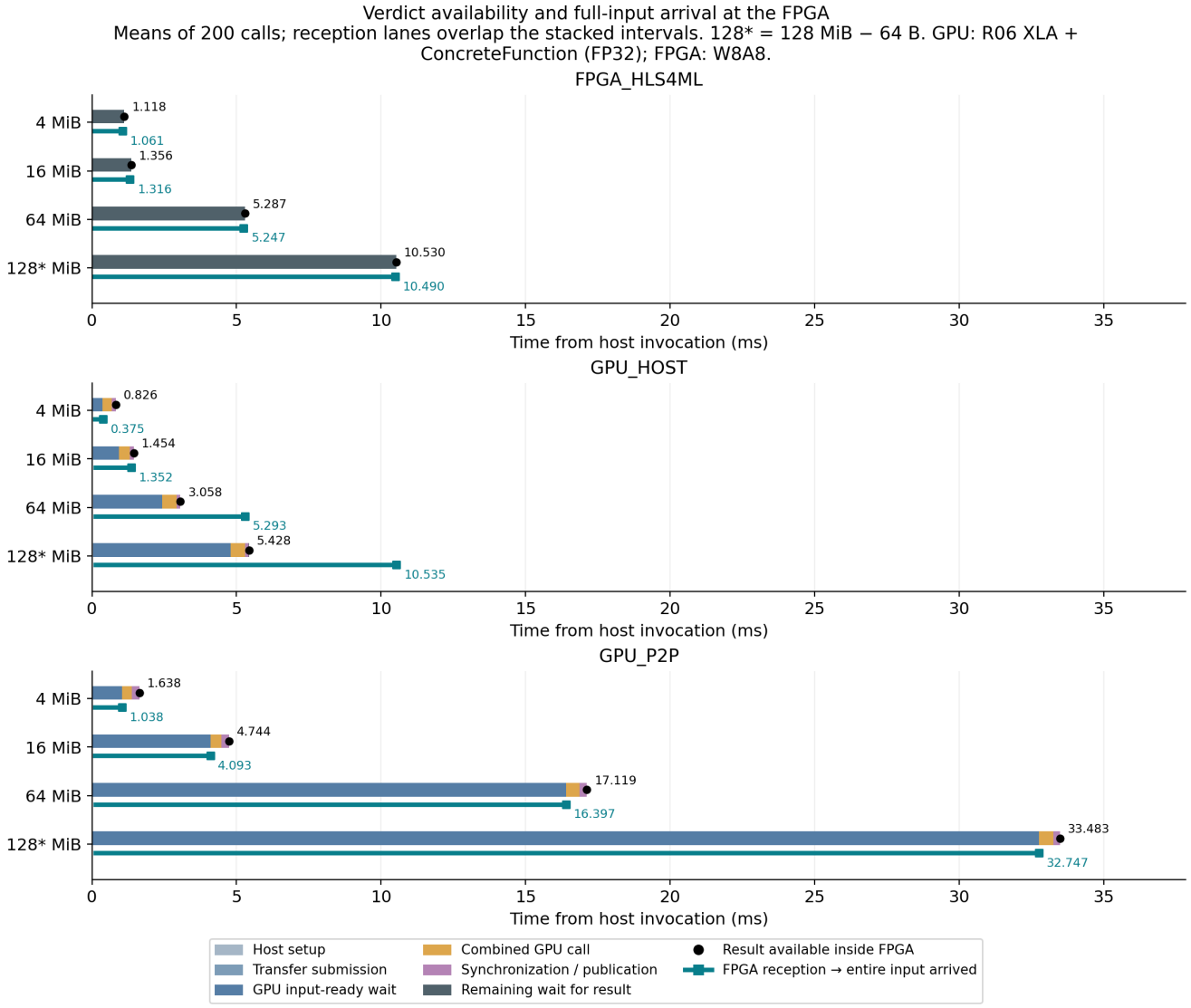


Solid arrows carry input; green arrows carry the classification result; dashed arrows denote control or polling. The primary endpoint is acceptance of the result inside the FPGA. P2P uses peer transactions through the PCIe fabric, not a dedicated FPGA-GPU cable.

Both GPU paths use the same R06 FP32 runtime; its preprocessing and CNN execute as one compiled graph. The FPGA checker consumes pixels while input arrives. GPU_HOST starts GPU work after its GPU copy completes; the independent FPGA copy may continue. GPU_P2P forwards input incrementally through a 2 KiB FIFO, then polls GPU memory and retrieves the published result.

Verdict availability and full-input arrival

Figure 4 · FPGA_HLS4ML, GPU_HOST and GPU_P2P



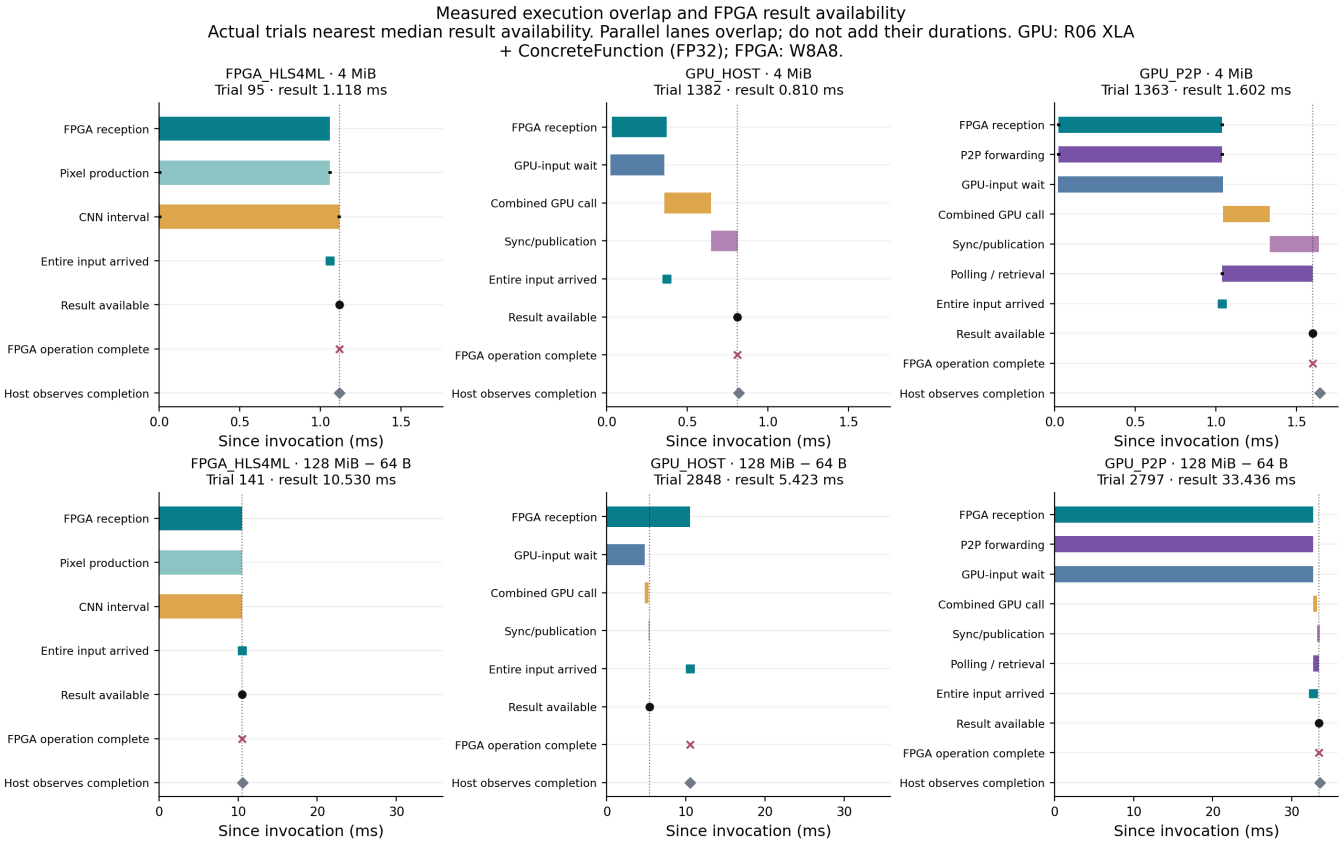
Stacked bars end at **mean classification-result availability** (black circle). Each separate teal lane spans the first through last FPGA payload beat; the square marks when the entire input has arrived. These intervals overlap and must not be added. All panels use the same time scale.

The stacked segments are consecutive observed host intervals, clipped at the FPGA result timestamp. They describe elapsed time, not isolated device costs. The combined GPU call covers framework dispatch and any execution completed before the call returns; remaining execution may appear in synchronization/publication. FPGA_HLS4ML has no GPU phases: its remaining wait includes streaming preprocessing and CNN execution. GPU-input wait ends at GPU readiness, not necessarily FPGA input completion.

For the largest GPU_HOST input, the result is accepted at 5.428 ms on average; the last FPGA input beat arrives at 10.535 ms. The 5.107 ms difference is remaining input reception, not verdict-notification overhead.

How the execution paths overlap

Figure 5 · Representative measured trials, including GPU_P2P



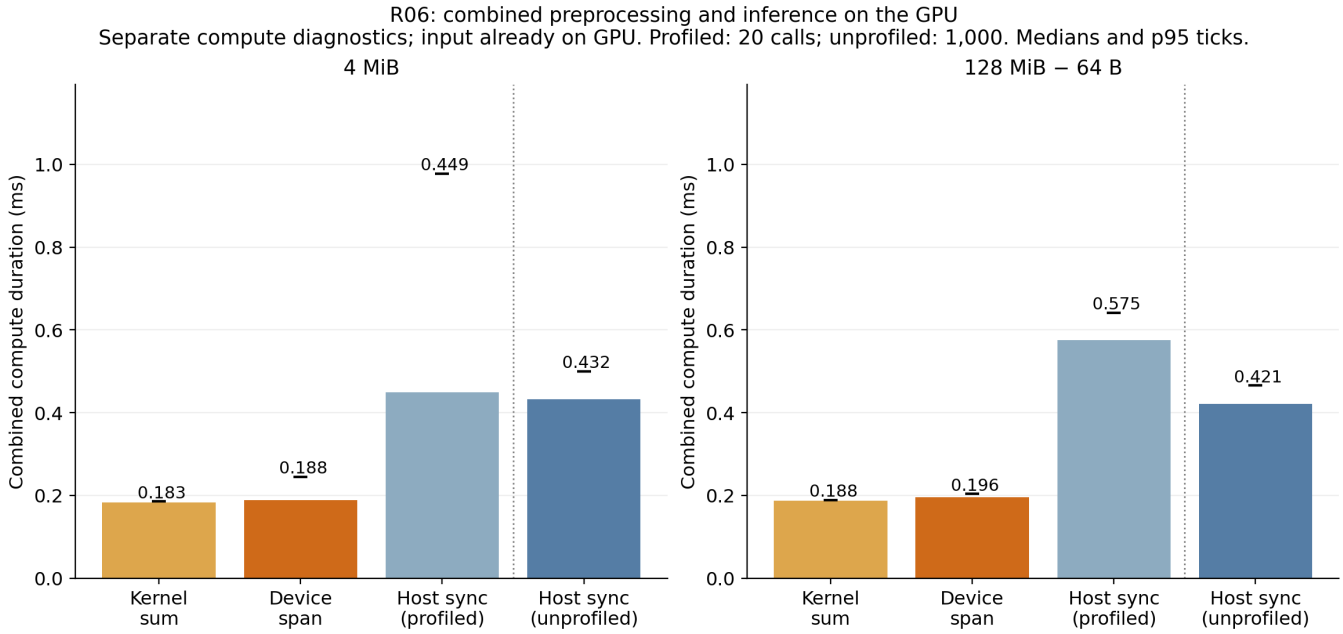
Each panel uses a real trial nearest that implementation's median result-availability latency. Its ID appears in the title. The two input-size rows share scales across implementations. Circle: result accepted. Square: entire input received. Cross: FPGA operation complete. Diamond: host observes completion.

GPU_P2P's polling/retrieval lane starts at the first readiness read and ends when the validated result reaches the controller. It can overlap GPU execution. FPGA forwarding and reception also overlap. GPU_HOST's result can arrive before its FPGA input transfer finishes.

FPGA event endpoints use the existing host/FPGA clock calibration; black endpoint marks indicate alignment uncertainty. Host-aligned GPU copy start was not recorded, so the figure shows the observed GPU-input wait. Lanes overlap and must not be added.

R06: compiled GPU preprocessing and inference

Figure 6 · Combined compute diagnostics, separate from board timings



Kernel sum: attributed GPU kernel durations. **Device span:** first kernel start through last completion, including gaps. **Host synchronized:** the complete call through scalar synchronization. The first three bars use 20 profiled calls; the fourth uses 1,000 unprofiled calls. Black ticks mark p95. These are alternative timing scopes, not additive stages.

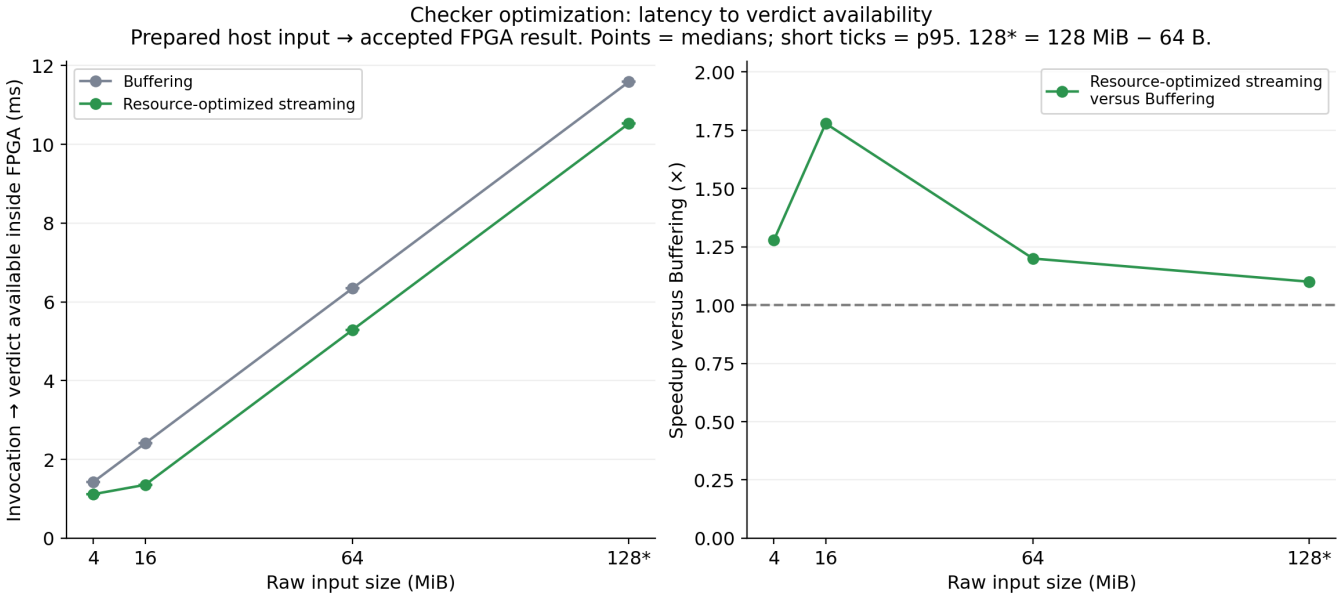
R06 uses one **tf.function(jit_compile=True)** for sampling, normalization and the original FP32 model. A cached **get_concrete_function()** is called directly for each input length. Joint traces contain 23 kernels. Preprocessing and inference cannot be separated faithfully without changing this fused execution.

Unprofiled compute medians are **0.432 ms at 4 MiB** and **0.421 ms at the largest input**. GPU device spans are 0.188 and 0.196 ms. Input transport, result publication and FPGA polling are outside these standalone measurements.

These diagnostics used CPU0-7 / NUMA0; all board-path measurements used CPU64 / NUMA1. Do not subtract these compute durations from Figures 1-5. First calls include approximately 137 ms of setup/compilation for these two examples and are excluded from steady-state timing; unseen input lengths need preparation. R07-R10 and DMA32 transport experiments are excluded.

Checker optimization: latency impact

Figure 7 · Buffering versus Resource-optimized streaming



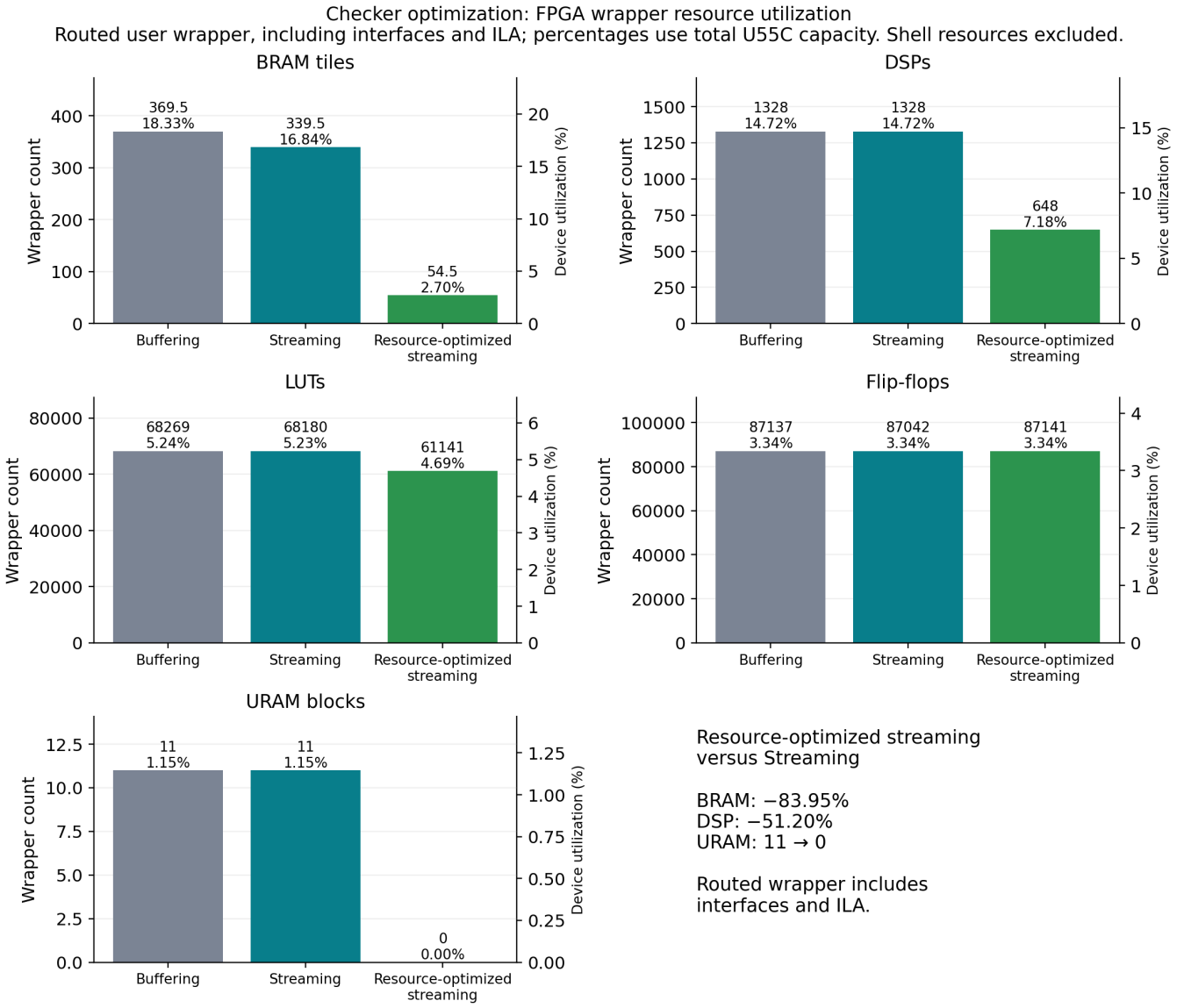
At 4 MiB, Buffering reaches its result in 1.429 ms and Resource-optimized streaming in 1.118 ms: 1.28x speedup. At the largest input, the speedup is 1.10x.

The left panel shows invocation-to-result medians and p95 ticks. The right divides Buffering median by Resource-optimized streaming median. Lines connect measured sizes only. Both use the same CSR14 endpoint as Figures 1 and 2.

Resource-optimized streaming combines native hls4ml FIFO optimization (profiling capacity 1024) with convolution reuse factors 72/72/288/288 for conv3/4/5/6. The profiling capacity is not a uniform final FIFO depth. The fixed quantized model, streaming preprocessing and timestamp interface are retained.

Checker optimization: resource impact

Figure 8 · FPGA variants only

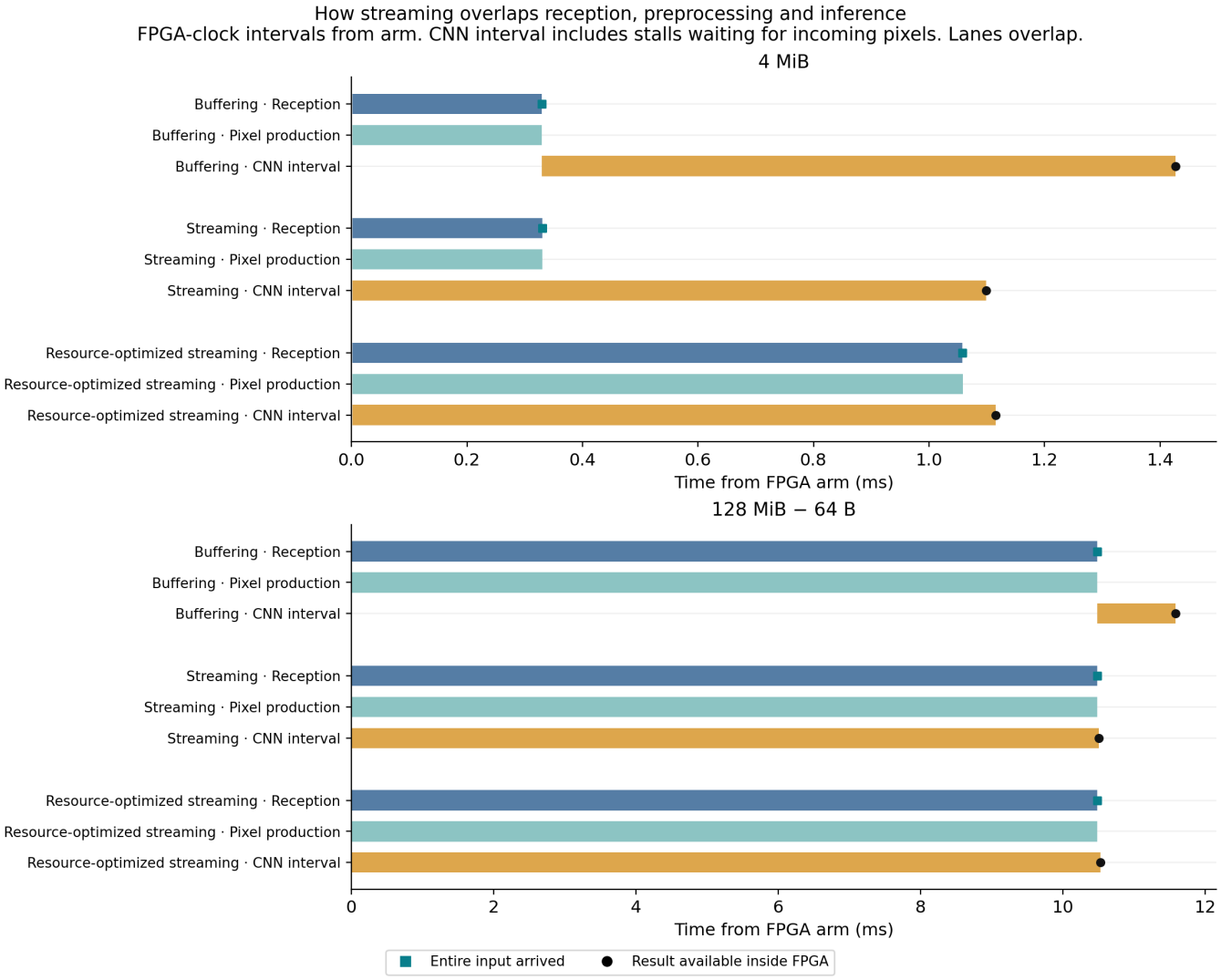


Relative to Streaming, Resource-optimized streaming reduces wrapper BRAM from **339.5 to 54.5 tiles** (83.95%) and DSPs from **1328 to 648** (51.20%). URAM falls from 11 blocks to zero.

Counts are routed user-wrapper resources, including interfaces and ILA. Right axes and bar labels express percentages of total U55C capacity. The shell occupies additional resources. These resource measurements are unchanged from the previous report.

How streaming overlaps the work

Figure 9 · Common FPGA timestamps across three checker variants



Buffering waits for the entire input and pixel frame before CNN consumption. Streaming starts the CNN as pixels become available. Resource-optimized streaming preserves this overlap while reducing memory and multiplier resources.

Squares mark full-input arrival; circles mark classification-result availability. Each example is the trial nearest median result-availability latency. All lanes use the FPGA clock relative to arm. The CNN interval includes waiting for pixels; it is not standalone arithmetic time. Trial IDs and clock uncertainty are exported.

Measurement definitions and evidence

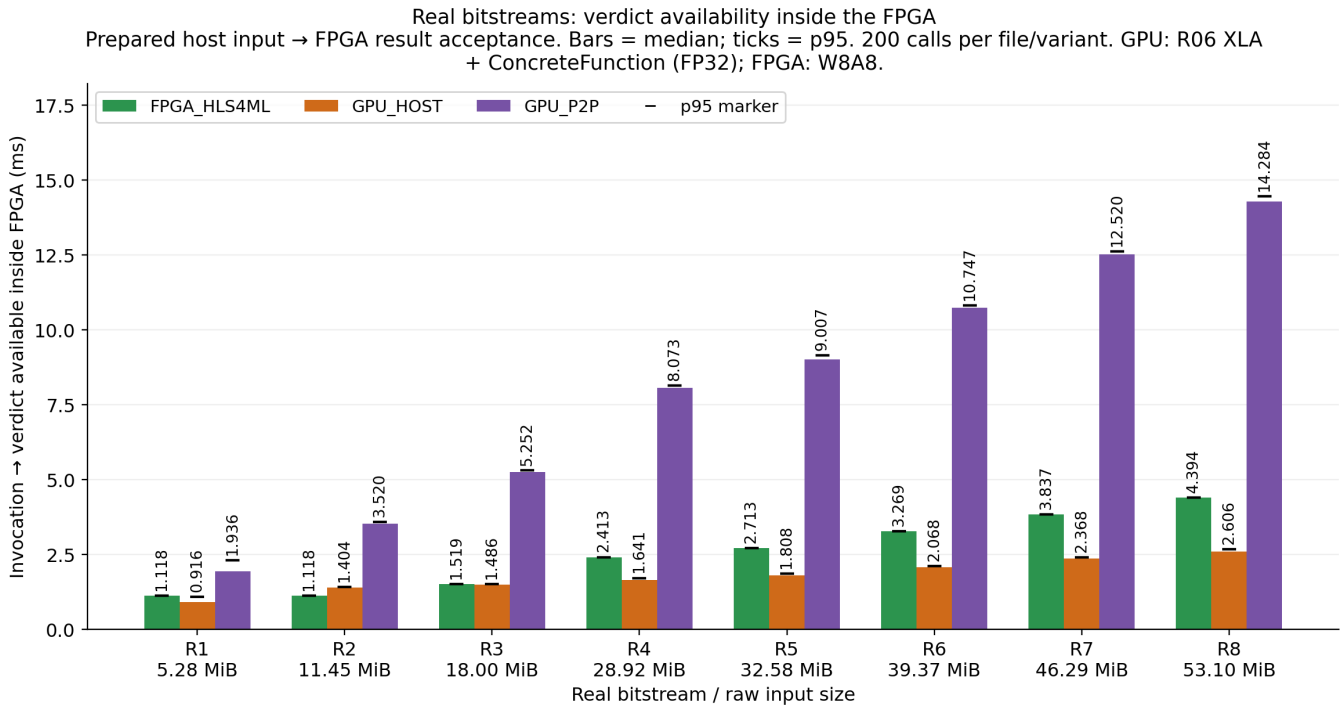
Item	Definition / scope
Primary endpoint	Prepared host invocation to accepted classification result inside FPGA: CSR14.
Input reception	CSR8 to CSR9; invocation to CSR9 separately records full-input arrival.
Final completion	CSR15 marks committed verdict/done. Host observation is later and remains in exported data.
GPU_P2P endpoint	CSR14 follows autonomous mailbox polling, payload retrieval and validation.
Not an early-release claim	The accepted score is available internally; the public verdict register may still wait for full input. No PR admission behavior was changed.
Workloads	4 synthetic sizes and 8 real files; 200 measured calls per implementation/input; warmups excluded.
FPGA labels	Buffering = OPT_0; Streaming = OPT_1; Resource-optimized streaming / FPGA_HLS4ML = OPT_6.
GPU labels	GPU_HOST = OPT_7; GPU_P2P = OPT_8. Original FP32 TensorFlow model; combined XLA graph with direct ConcreteFunction calls (R06).
Hardware	U55C device2 at 81:00.0; MI210 GPU0 at 03:00.0; hacc-gpu-u55c-01. CPU64 / NUMA1; GPU on NUMA0.
Clock alignment	Existing per-block affine calibration; midpoint estimates with exported lower/upper bounds. Maximum endpoint half-width: 0.565 us.
Correctness / stability	Accepted campaigns: per-image admission, exact FPGA logits, GPU CPU-reference parity, autonomous P2P polling and baseline-drift gates.
Cross-campaign comparison	FPGA results retained from preprocessing-v2 campaign; GPU results selected from accepted R06 hardware campaign. Shared OPT_1 resident host-completion medians differ by at most 0.027%. These are independent runs, not paired trials.
Compilation / diagnostics	Known input lengths compiled and warmed before board arming. Figure 6 uses separate joint R06 diagnostics on CPU0-7/NUMA0.
Comparability limits	FPGA W8A8 versus GPU FP32; one machine/device pairing and fixed run order. No cold-SSD, equal-precision or power claim.

This is offline reanalysis of existing measurements. No FPGA builds, board programming or benchmark reruns were performed. All 12,000 selected calls pass the new event and calibration checks; original host-completion statistics are reproduced unchanged.

Reproduce from the repository root: **ml_baseline/.venv/bin/python hls4ml/optimize/reports/optimizations/2026-10-22_gpu_opt_xla_concretefunction/build_report.py**. The folder includes standalone titled figures, per-trial event data, statistics, timing tests and an evidence manifest.

Appendix A1 · Real-bitstream verdict availability

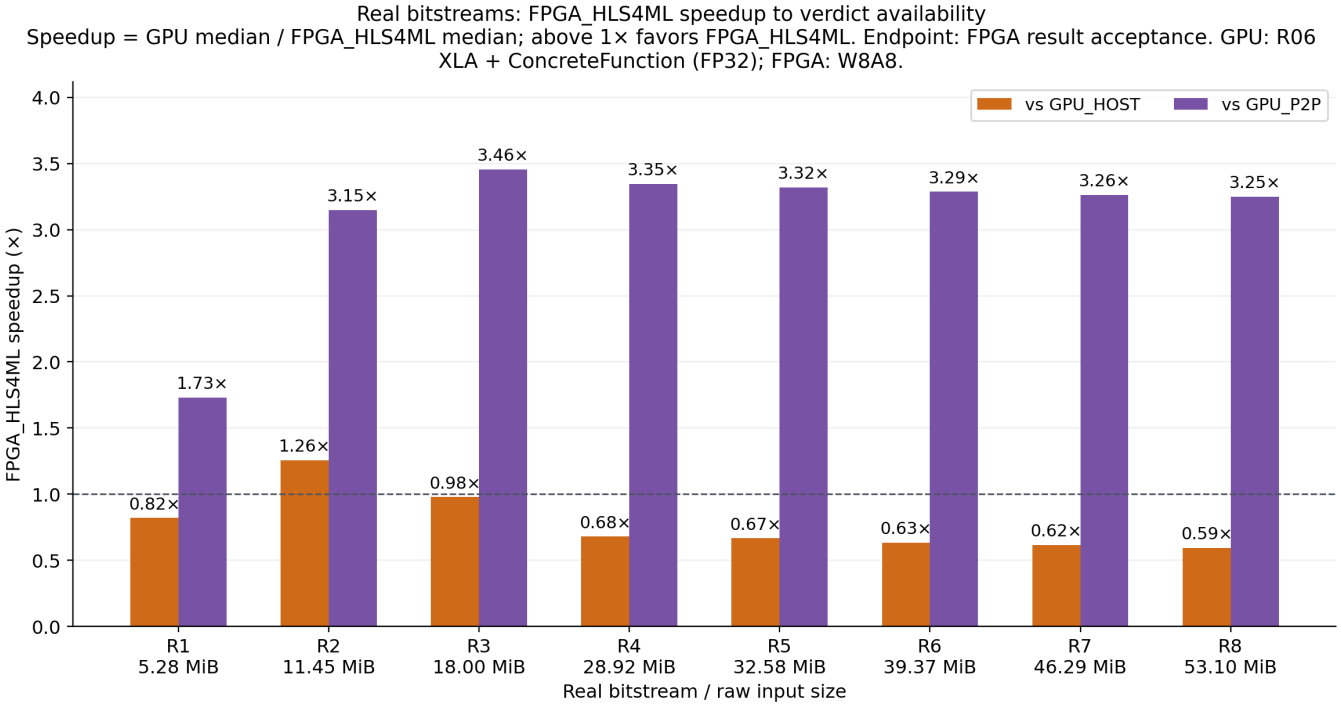
Eight real files, ordered by size. The endpoint matches Figure 1: host invocation to accepted classification result inside the FPGA. File reading and input preparation are outside the interval.



Each group compares FPGA_HLS4ML, GPU_HOST and GPU_P2P for the same file. Labels show medians; black ticks show p95. Different files also have different sizes, so this does not isolate content-dependent effects.

Runtime and transport effects can make results non-monotonic between files. The measurements do not establish that real bitstream contents inherently require more computation.

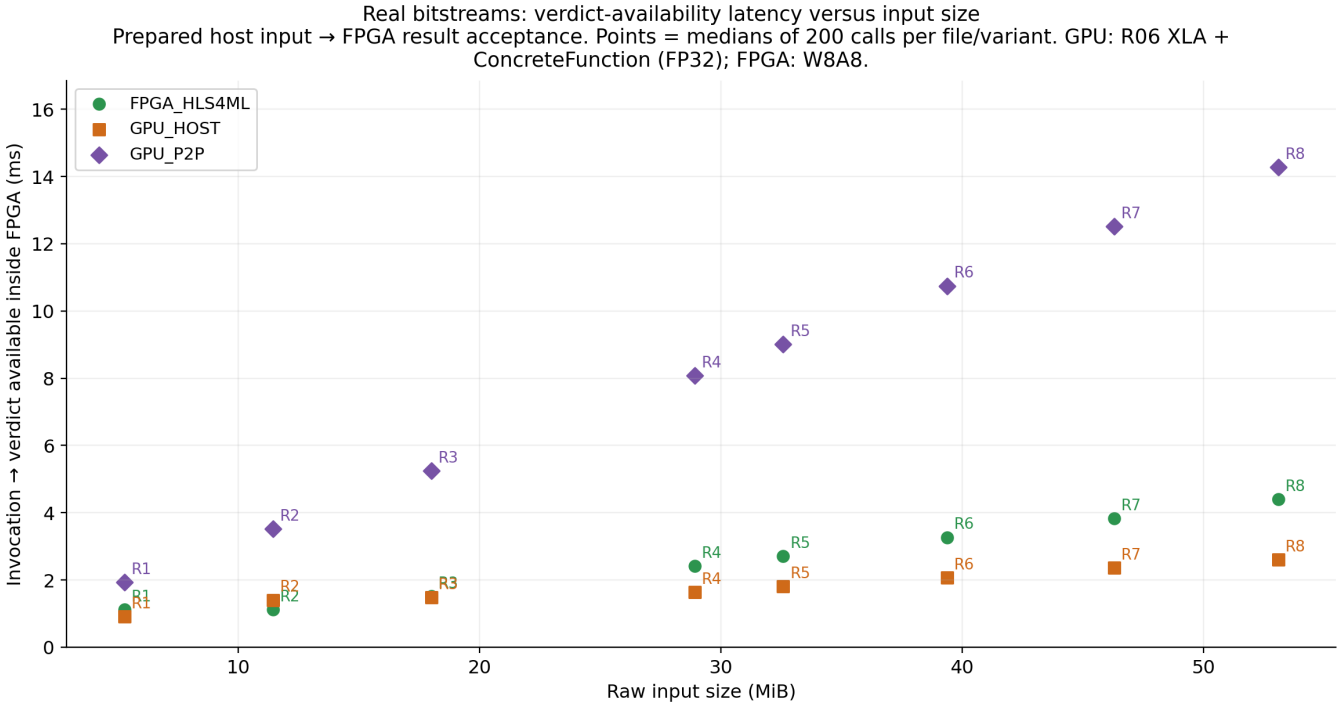
Appendix A2 · Real-bitstream checker speedup



For each file, divide the GPU median invocation-to-result latency by the FPGA_HLS4ML median using the same endpoint. Ratios above 1 favor the checker; ratios below 1 favor the GPU path. Inputs and accepted timing scope match.

This report retains per-file speedup bars and does not include a speedup scatter plot.

Appendix A3 · Real-bitstream latency by size



Linear size axis; unconnected median result-availability latencies. R1-R8 identify the files below. No fitted trend or synthetic interpolation is overlaid.

ID	Bytes / MiB	Source identifier
R1	5,540,996 / 5.28	HR/build/bitstreams/config_0/vfpga_c0_0.bin
R2	12,004,440 / 11.45	H4/build/bitstreams/config_0/vfpga_c0_0.bin
R3	18,876,484 / 18.00	H16/build/bitstreams/config_0/vfpga_c0_0.bin
R4	30,323,588 / 28.92	full_dataset_it1/artifacts/bitstreams/config_027/vfpga_c027_0.bin
R5	34,162,708 / 32.58	full_dataset_it1/artifacts/bitstreams/config_109/vfpga_c109_0.bin
R6	41,278,872 / 39.37	full_dataset_it2/artifacts/bitstreams/config_105/vfpga_c105_0.bin
R7	48,540,364 / 46.29	full_dataset_it4_big_ro/artifacts/bitstreams/config_050/vfpga_c050_0.bin
R8	55,676,772 / 53.10	full_dataset_it4_big_ro/artifacts/bitstreams/config_075/vfpga_c075_0.bin